SECURITY



Adam’s Bridge v3.0: Architectural Masking for PQC

Mojtaba Bisheh-Niasar *Principal Security Architect, Microsoft*

Kiran Upadhyayula *Senior Hardware Engineer, Microsoft*



Adams Bridge Roadmap

1.0 (APR 2025)

ML-DSA Accelerator

Streaming msg support

2.0 (OCT 2025)

1.0 bug fixes

ML-KEM Accelerator

Extended Masking on ML-DSA

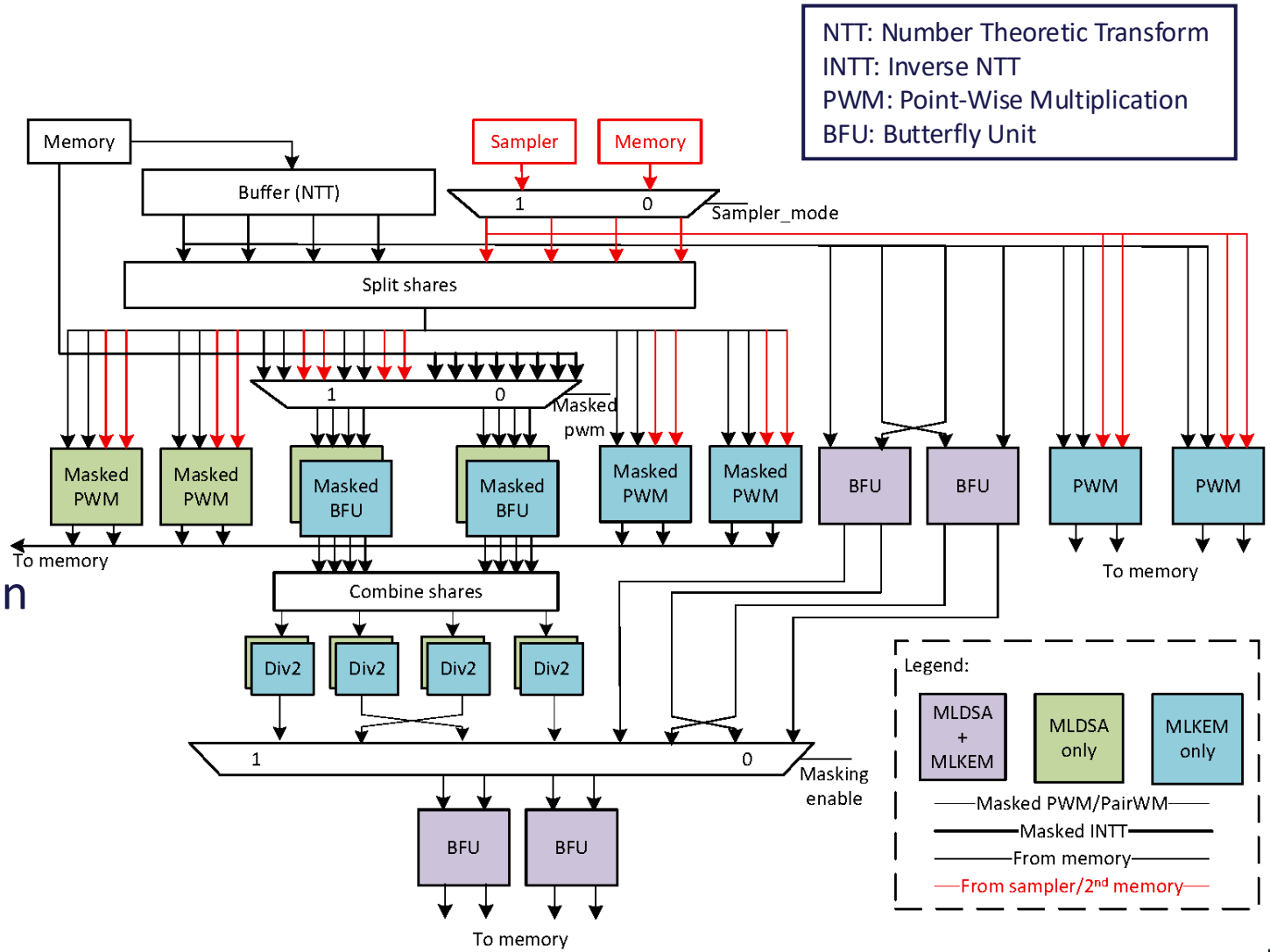
External_Mu support for ML-DSA

3.0 (OCT 2026)

Optimized ABR with reduced area and enhanced masking techniques

Recall: ABR 2.0 NTT

- Shuffled Point-Wise Mult/Add/Sub
- Shuffled Number Theoretic Transform (NTT)
- Masked PWM (Point-Wise Mult)
- Hybrid approach for Inverse NTT (INTT)
  - Masked 1st stage
  - Subsequent stages of INTT operation are only shuffled
- 4x latency overhead

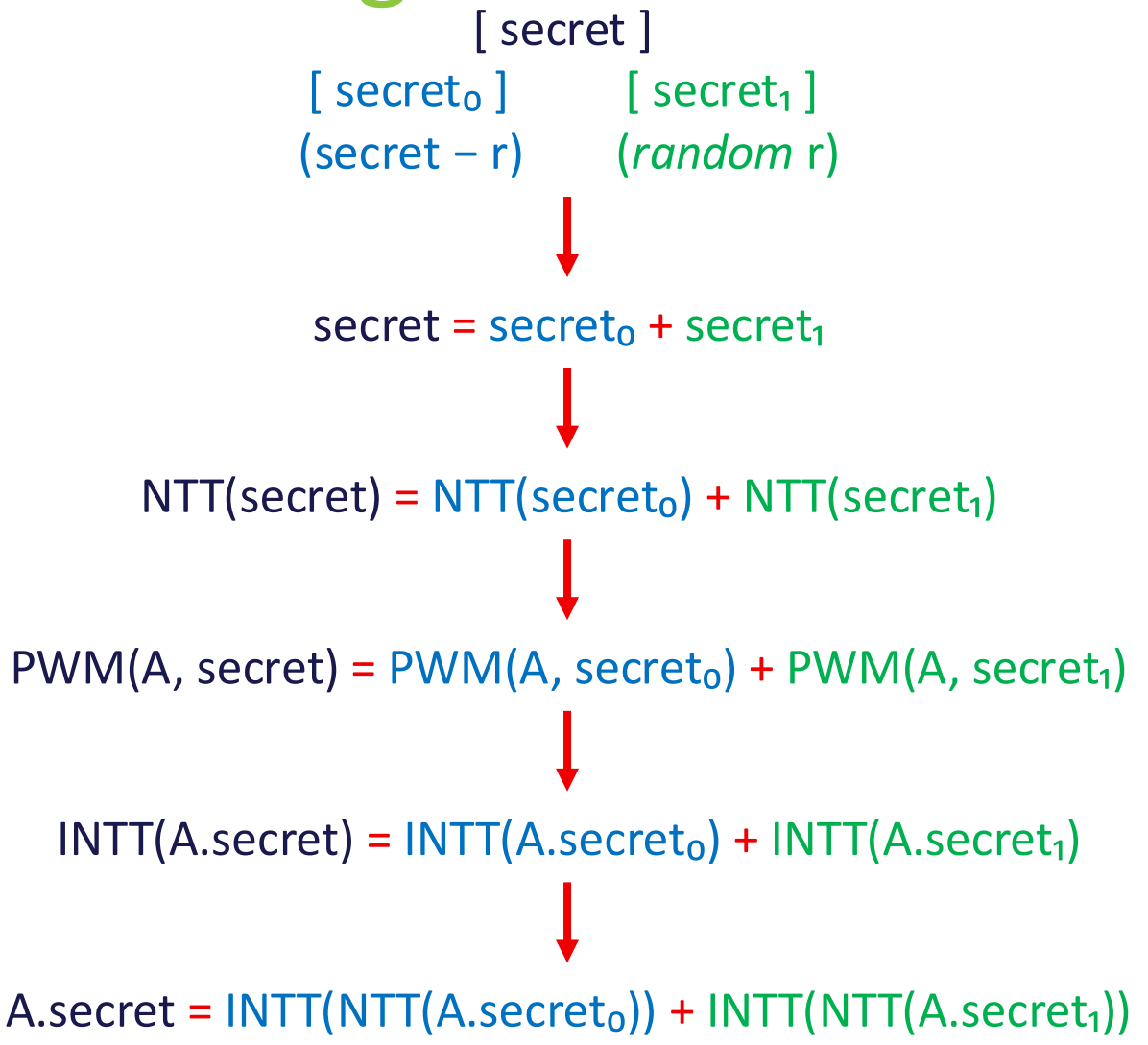


ABR 2.0: feedbacks from ecosystem

- DPA (Differential Power Analysis) attack on the 1st layer of INTT(c, s1) and INTT(c,s2)
  - The challenge c is a known value
  - formulate a hypothesis on the power consumption
- DPA attack on B2A (Boolean to Arithmetic Conversion)
  - Glitch attack on the countermeasure
- DPA attack on MLDSA PWA: y+cs1
  - hypothesis for cs1, where most of coeffs are set to zero
- CCA (Chosen-Ciphertext Attack) on MLKEM decapsulation PWM(s1, u)
  - choose u that produces zeros in specific positions in NTT domain
- DPA attack on MLKEM PWM(s1, u)
  - Attack on the combined share input to the Barrett reduction

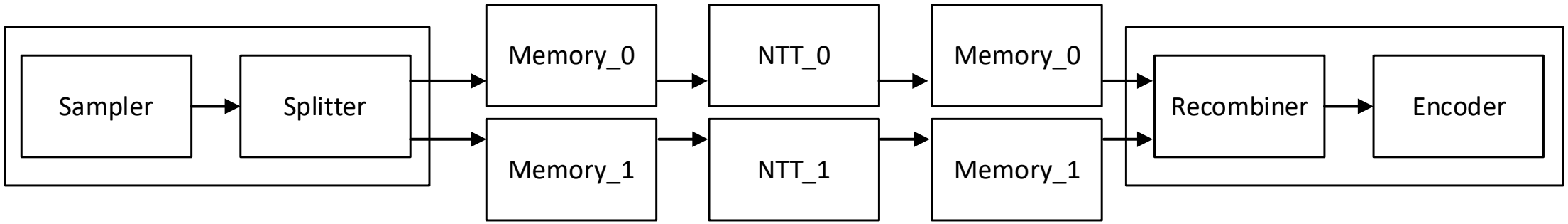
ABR 3.0: Architectural level masking

- **Classic Masking (ABR 2.0)**
  - Protects arithmetic operations such as adders, multipliers, and modular reductions.
  - Requires masked implementations of nonlinear operations (e.g., AND, shifts, subtraction, and modular reduction).
  - Incurs significant area and performance overhead.
- **Architectural-Level Masking**
  - Exploits the linearity of the transform.
  - NTT is linear: **NTT(a + b) = NTT(a) + NTT(b)**.
  - PWM/INTT is linear as well.
  - Input shares can be masked before the NTT and processed independently, using unprotected NTT datapath.



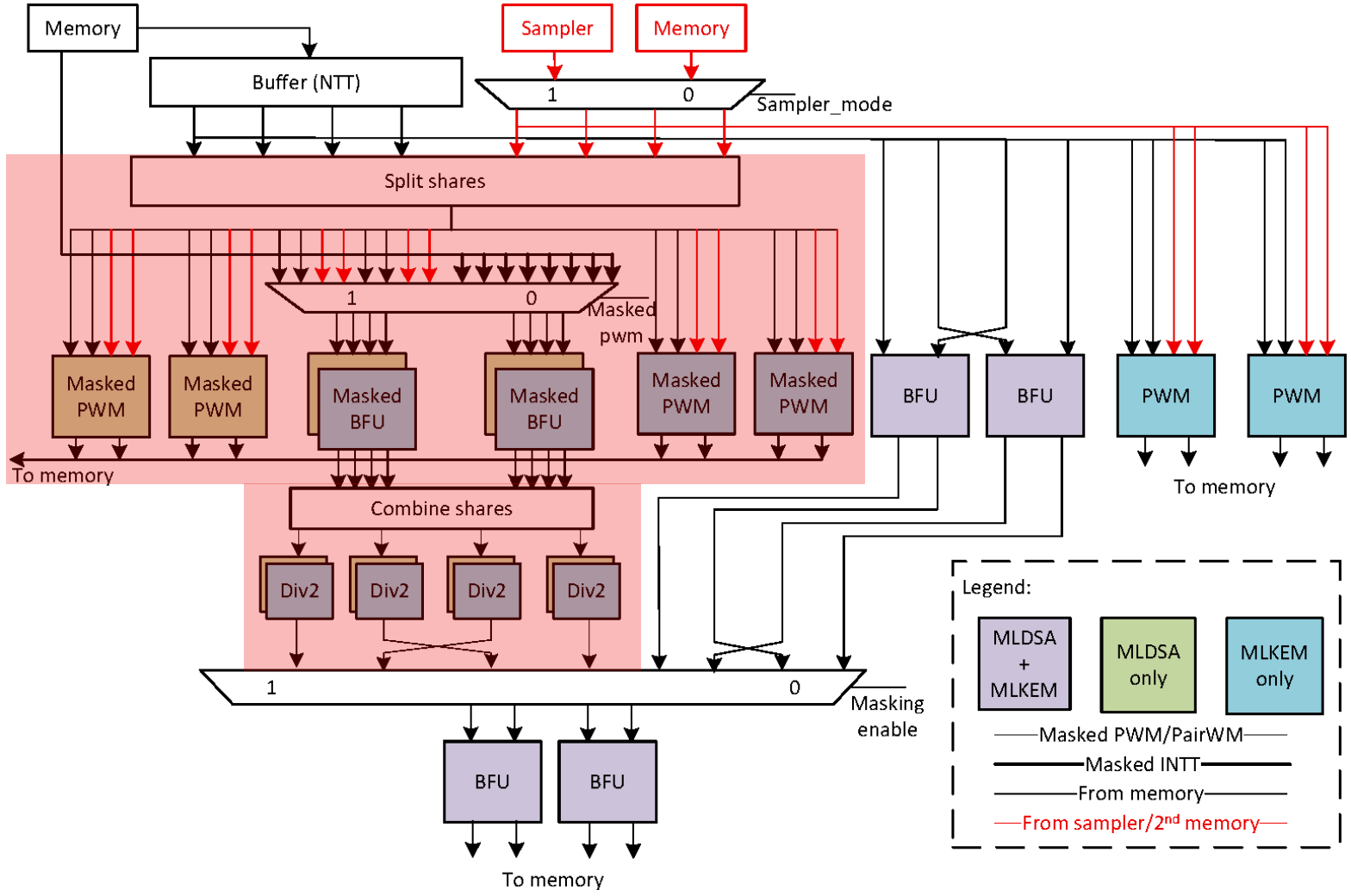
ABR 3.0: Parallel NTT units

- Randomly split the input polynomial into two shares before storing them into memory.
- Each share is processed independently through the existing unprotected NTT/INTT/PWO datapath.
- No masking logic is added inside the arithmetic units, avoiding datapath-level overhead.
- The two shares remain separated throughout the transform operations to preserve masking.
- Shares are combined only after the sensitive computation is completed.
- Enables reuse of the existing high-performance NTT engine with minimal area impact.



Area Reduction in ABR v3.0

Removing datapath-level masking overhead.



Memory Layout

Instance	**ABR 2.0**	**ABR 3.0** Without Masking	**ABR 3.0** With Masking	**Masking Overhead** (compared to ABR 2.0)
abr_sk_mem_bank0/1	2 × 2,384 B	2 × 2,384 B	2 × 2,384 B	No change
abr_w1_mem	256 B	256 B	256 B	No change
abr_mem_inst0_bank0/1	2 × 9,984 B	2 × 9,600 B	2 × 19,200 B	**1.92× larger**
abr_mem_inst1	6,912 B	768 B	1,536 B	**78% reduction**
abr_mem_inst2	17,664 B	18,432 B	36,864 B	**2.1× larger**
abr_mem_inst3	3,072 B	Removed	Removed	**Eliminated**
abr_sig_z_mem	4,480 B	4,480 B	4,480 B	No change
abr_pk_mem	2,560 B	2,560 B	2,560 B	No change
Total SRAM	**59.7 KB**	**49.2 KB**	**86.7 KB**	**+27 KB (+45%)**

Area Comparison

- **ABR 2.0**
  - Protected ABR (ML-DSA-87 + ML-KEM-1024): 0.1096 mm² @ 5 nm
    - **0.0860 mm²** standard-cell logic
    - **0.0236 mm²** SRAM (59 KB)
  - Around 45% of total Caliptra v2.1
- **ABR 3.0**
  - Highly Protected ABR (ML-DSA-87 + ML-KEM-1024): 0.0459 mm² @ 5 nm
    - **0.0123 mm²** standard-cell logic → **80%** standard-cell area reduction
    - **0.0335 mm²** SRAM (86 KB) → 27 KB SRAM overhead
  - **58%** overall area reduction

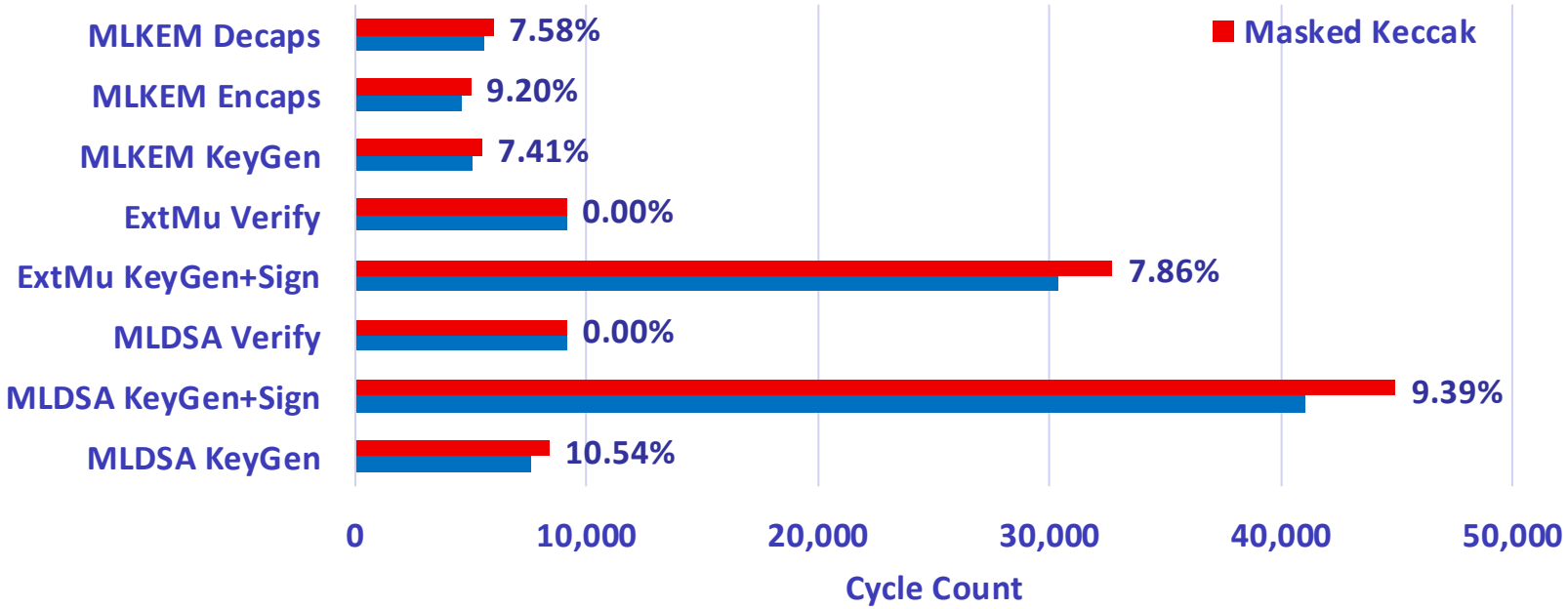
Performance Comparison

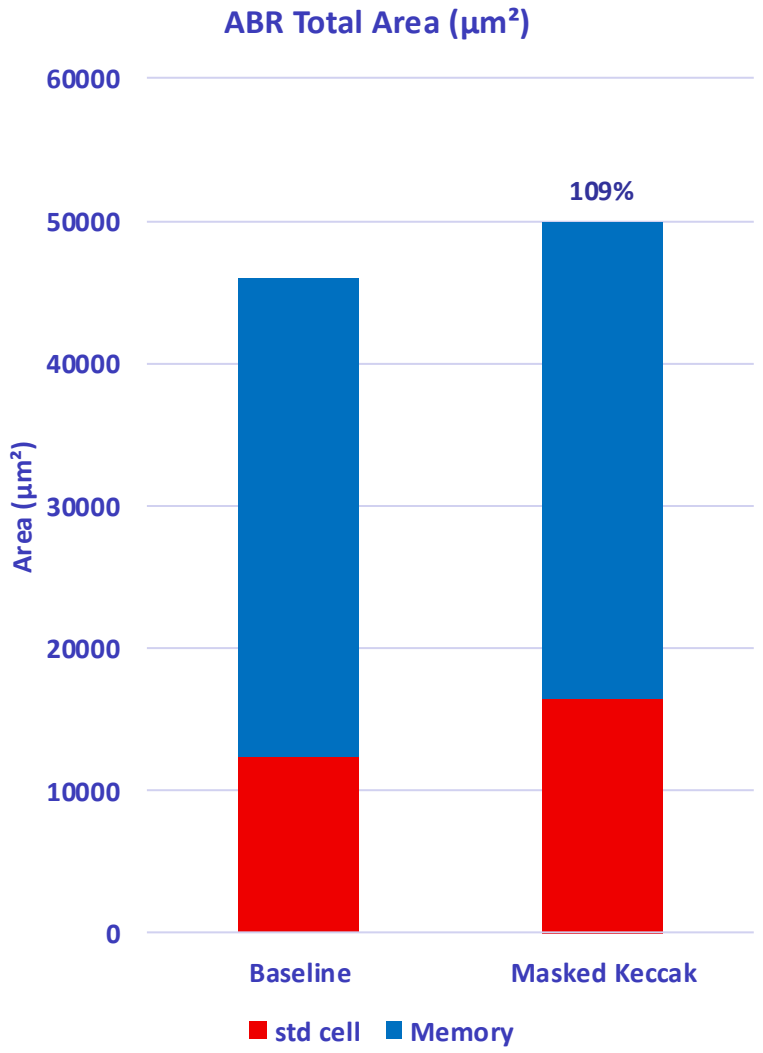
ML-DSA-87	ABR 2.0 (Cycle Count)	ABR 3.0 (Cycle Count)	Improvement
KeyGen	15,600	15,154	**2.9%**
Sign (1 round)	36,700	34,928	**4.8%**
Sign (Avg., 3.85 rounds)	141,295	134,473	**4.8%**

ML-KEM-1024	ABR 2.0 (Cycle Count)	ABR 3.0 (Cycle Count)	Improvement
KeyGen	6,603	6,640	**-0.6%**
Encapsulation	8,509	7,930	**6.8%**
Decapsulation	12,117	11,056	**8.8%**

Keccak Masking

- Area overhead (~4x)
- Each Keccak includes 24 internal rounds
- Current arch: 2 rounds per clk
- Masking drops performance to 4 clk per round (8x slower)





High-Level Flow

- Highlighted operations represent the masked operations in ABR 2.0.
- ABR 2.0 provides full PWM masking and partial INTT masking.
- ABR 3.0 expands masking coverage to protect more operations across the algorithm flow.

Current Flow	Optimized Flow
μ ←H(tr ||M,512)	
ρ'←H(K||rnd||μ,512)	
(ρ,K,tr,s1,s2,t0)←skDecode(sk)	
	t0_0, t0_1 ← split (t0)
	s1_0, s1_1 ← split (s1)
	s2_0, s2_1 ← split(s2)
ˆt0 ←NTT(t0)	ˆt0_0 ←NTT(t0_0)
	ˆt0_1 ←NTT(t0_1)
sˆ1 ←NTT(s1)	sˆ1_0 ←NTT(s1_0)
	sˆ1_1 ←NTT(s1_1)
sˆ2 ←NTT(s2)	sˆ2_0 ←NTT(s2_0)
	sˆ2_1 ←NTT(s2_1)
y ←ExpandMask(ρ' ,κ)	
	y_0, y_1 ← split(y)
NTT(y)	NTT(y_0)
	NTT(y_1)
Aˆ ←ExpandA(ρ) AND Aˆ ◦NTT(y)	Aˆ ←ExpandA(ρ) AND
	Aˆ ◦NTT(y_0) AND Aˆ ◦NTT(y_1) <No Shuffle>
w ←NTT−1(Aˆ ◦NTT(y))	w_0 ←NTT−1(Aˆ ◦NTT(y_0))
	w_1 ←NTT−1(Aˆ ◦NTT(y_1))
	w ← w_0 + w_1
(w1,w0) ←Decompose(w) AND c˜←H(μ ||w1Encode(w1),2λ)	
c ←SampleInBall(c˜)	
cˆ ←NTT(c)	
κ ←κ +ℓ	
cˆ◦ sˆ1	cˆ◦ sˆ1_0
	cˆ◦ sˆ1_1
cs1←NTT−1(cˆ◦ sˆ1)	cs1_0←NTT−1(cˆ◦ sˆ1_0)
	cs1_1←NTT−1(cˆ◦ sˆ1_1)
z ←y +cs1	z← y_0 + y_1 + cs1_0+ cs1_1
||z||∞ ≥ γ1 −β	
SigEncode(z)	
cˆ◦ tˆ0	cˆ◦ tˆ0_0
	cˆ◦ tˆ0_1
ct0←NTT−1(cˆ◦ tˆ0)	ct0_0←NTT−1(cˆ◦ tˆ0_0)
	ct0_1←NTT−1(cˆ◦ tˆ0_1)
	ct0← ct0_0 + ct0_1
cˆ◦ sˆ2	cˆ◦ sˆ2_0
	cˆ◦ sˆ2_1
cs2←NTT−1(cˆ◦ sˆ2)	cs2_0←NTT−1(cˆ◦ sˆ2_0)
	cs2_1←NTT−1(cˆ◦ sˆ2_1)
r0 ←(w0 −cs2)	r0 ←(w0 −cs2_0- cs2_1)
h ←MakeHint(w1 ct0 cs2)	
||r0||∞ ≥ γ2 −β	
(r0,r1) ←cs2	

ABR protection

- Configurable protection based on application requirement

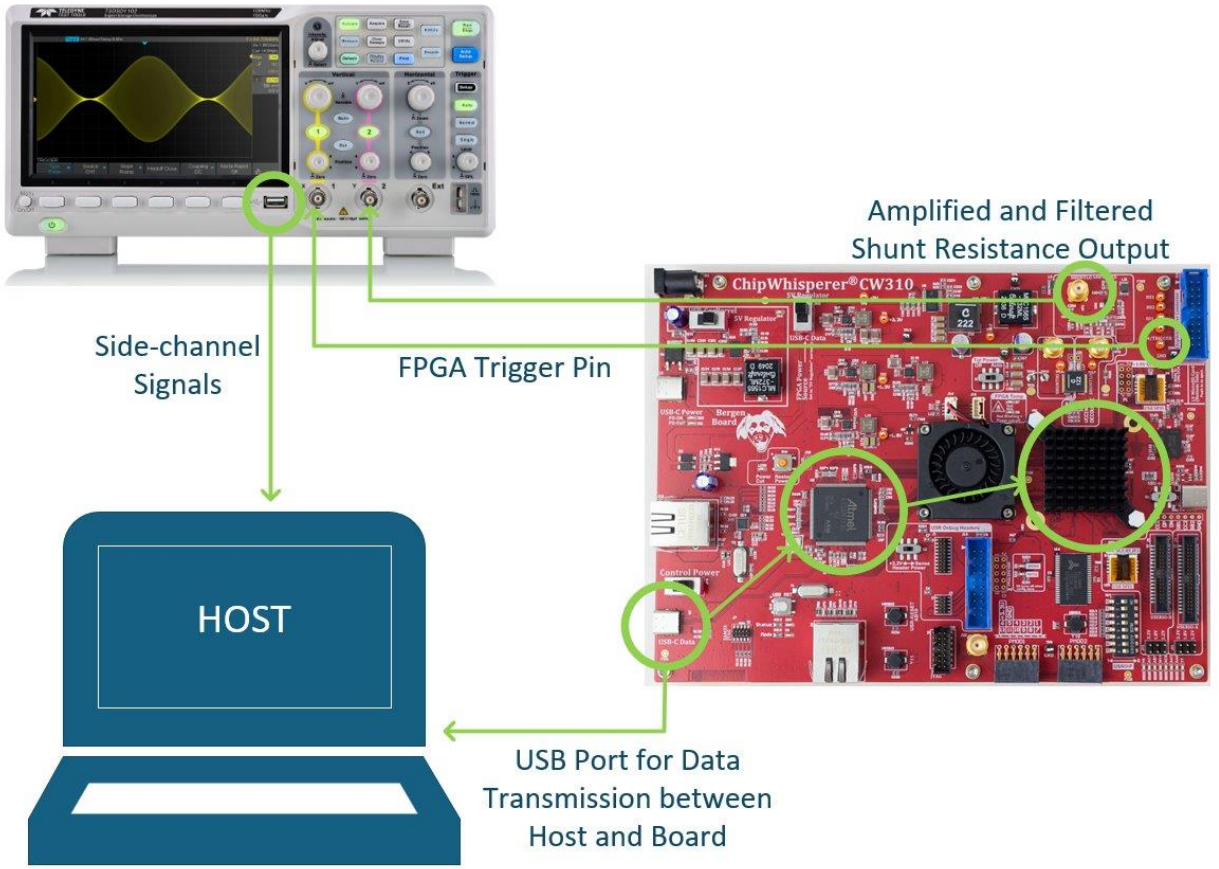
Masking Configuration	Protection Features	NTT/INTT/PWM Shuffling	NTT/INTT/PWM Masking	Keccak Masking
0b00	Shuffling Only	☑	☒	☒
0b01	Core Masking	☑	☑	☒
0b11	Extended Masking	☑	☑	☑

Security Enhancement

Issue identified in ABR 2.0	Resolution in ABR 3.0
DPA attack on the output of 1st INTT layer	☑ Fully Masked
DPA attack on B2A	☑ Removed
DPA attack on MLDSA PWA: y+cs1	☑ Fully Masked
CCA attack on MLKEM decapsulation PWM(s1, u)	☑ Fully Masked
DPA attack on MLKEM PWM(s1, u)	☑ Fully Masked

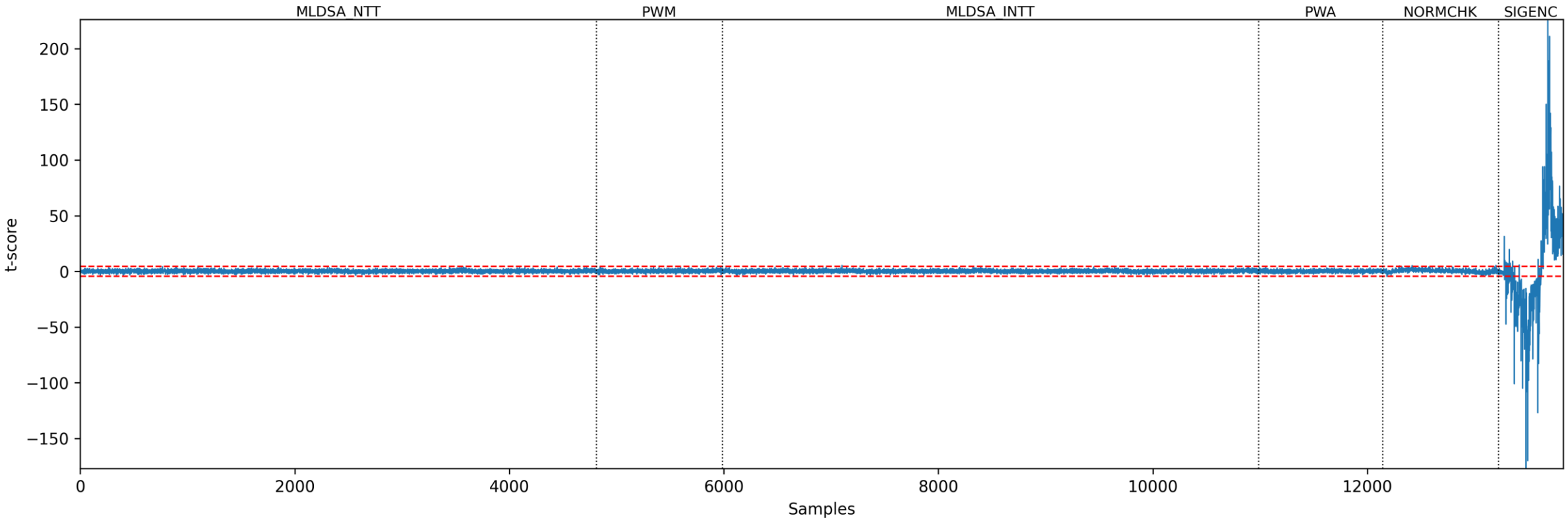
SCA Experimental Flow

- Programs the FPGA with the ABR bitstream
- Configures PLL to generate a 10 MHz clock
- Sets up the oscilloscope:
  - Defines buffer size N
  - Specifies trigger channel and capture channel for N samples
  - Configures FPGA sampling rate (156 MS/s)
- TVLA is a statistical method used to detect side-channel leakage using Welch's t-test



Adam’s Bridge v3.0 TVLA results

- The secret-involved operations (NTT/PWM/INTT/PWA) do not show any leakage.
- SIGENC is already a public value, so observing a high t-value there does not indicate a security leakage.



Call to Action

- ABR v3.0:
  - Optimized Area PQC accelerator with enhanced masking techniques
  - Embedded into Calitpra 2.2
- Integrate Adams Bridge in your ASIC
- Where to find additional information
  - Review and Feedback on Adams Bridge Architecture:
    - https://github.com/chipsalliance/adams-bridge
  - Caliptra
    - https://caliptra.io/
    - https://lists.chipsalliance.org/g/caliptra-wg/join
  - Join CHIPS Alliance
    - https://www.chipsalliance.org/join/

Thank You!

